

程序设计训练 OJ 在线评测系统

—— 课程大作业实验报告 ——

作业模块：基础模块 Step 1–6（30 分）+ 进阶模块
AI 智能命题（10 分）

技术栈：FastAPI（异步）+ Streamlit + JSON 文
件存储 + subprocess/psutil 评测 + DeepSeek 大
模型

代码规模：后端 17 个模块 2535 行 + 前端 1899 行
= 4434 行

git 提交：82 次 Conventional Commits 规范提交

报告生成于 2026 年 9 月

目录

1. 系统功能与设计
2. 关键实现与难点
3. 成果展示
4. AI 使用说明
5. 总结与建议

1. 系统功能与设计

1.1 系统目标

本系统实现了一个轻量级但功能完整的 **OJ (Online Judge) 在线评测系统**，覆盖题目管理、代码评测、用户与权限、评测日志、Streamlit 前端、**AI 智能命题**六大模块，**所有后端 API 均使用 FastAPI 异步接口 (`async def`)**，符合"不使用异步编程的无法拿到本次作业分数"的硬性要求。

1.2 系统架构

系统采用经典的 **前后端分离 + JSON 文件存储架构**，自上而下分为四层：

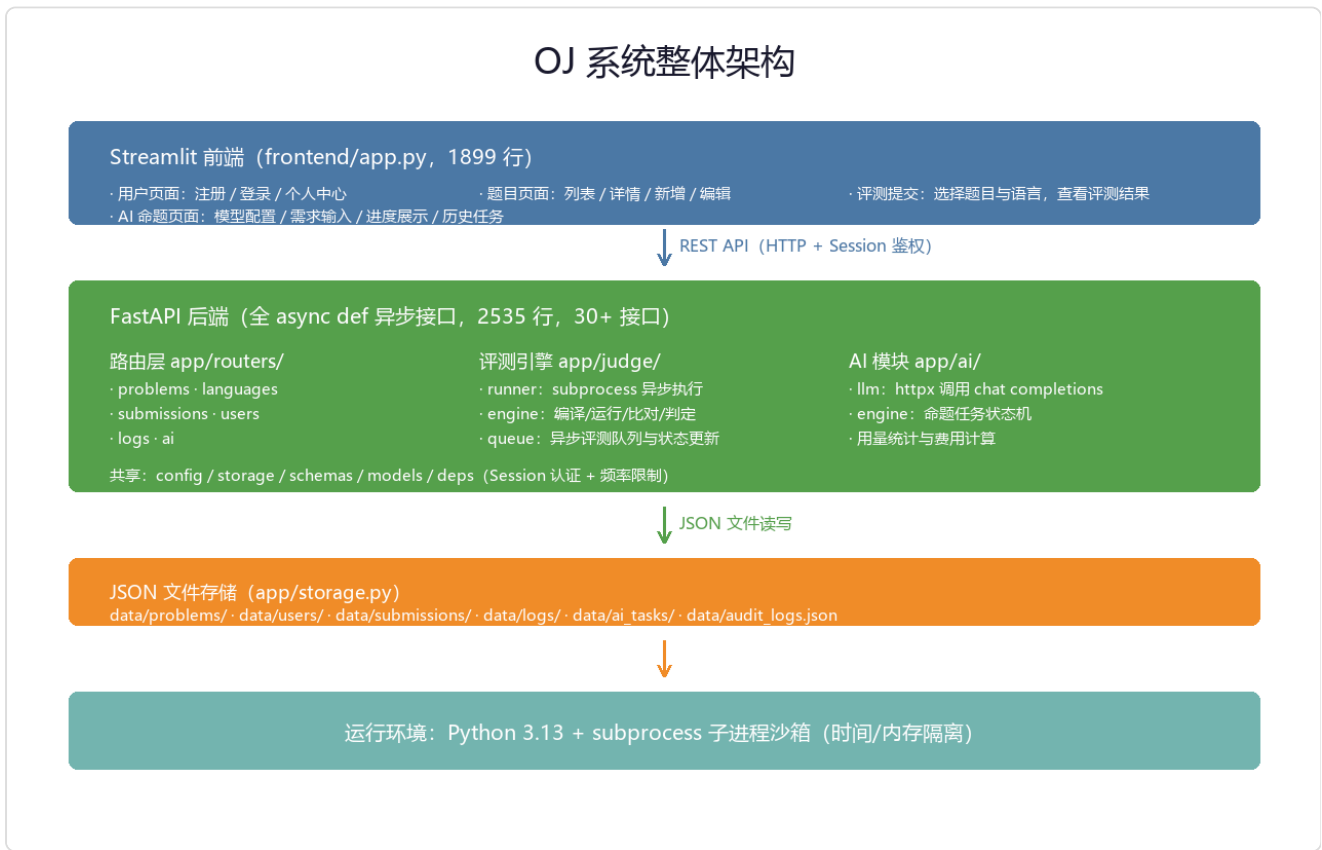


图 1.1 OJ 系统整体架构

前端层 (Streamlit, frontend/app.py, 1899 行)：通过 `requests` 库调用后端 REST API，不直接访问数据。包含用户、题目、评测提交、AI 命题四组页面，所有受保护操作均通过 Session Cookie 保持登录态，刷新页面通过 localStorage 恢复会话。

后端层 (FastAPI, app/, 2535 行, 30+ 个异步接口)：

路由层 `app/routers/`

六个路由模块，分别对应 6 个功能域。

评测引擎 `app/judge/`

runner / engine / queue 三层结构。

AI 模块 `app/ai/`

llm 模型调用 + engine 任务状态机。

共享基础设施

config / storage / schemas / models / deps
(认证 + 限流)。

存储层（ `app/storage.py` ）：所有数据落盘到 `data/` 目录下的 JSON 文件，包括题目、用户、提交、日志、AI 任务、AI 模型配置、审计日志。读写采用 `_read_json` / `_write_json` 统一封装，通过临时文件 + 原子 rename 避免半写状态。

运行环境：用户代码作为子进程拉起，**时间限制**通过 `asyncio.wait_for` 强制中断，**内存限制**通过 `psutil` 后台协程轮询 RSS 实现，跨平台兼容纯 Windows 与 Linux。

1.3 技术选型

维度	选型	理由
Web 框架	FastAPI	原生 async/await 生态、OpenAPI 自动文档、Pydantic 校验
前端框架	Streamlit	纯 Python、组件丰富、易于调用后端 API
存储	JSON 文件	符合"题目配置加载"要求；零外部依赖；透明可读
认证	SessionMiddleware + bcrypt	无需前端存 token；密码哈希安全
评测执行	subprocess + psutil	跨平台、无需 Docker；时间用 wait_for、内存用 psutil 轮询
LLM 调用	httpx	异步 HTTP 客户端，OpenAI 兼容 chat completions

1.4 模块划分

六大模块对应六个 `routers/` 子模块，文件归属严格对应：

评分模块	对应文件	关键接口	分值
------	------	------	----

Step 1 题目管理	<code>app/routers/problems.py</code> (168 行)	GET/POST/PUT/DELETE <code>/api/problems/</code>	5
Step 2 评测控制	<code>app/judge/</code> (480 行) + <code>routers/languages.py</code> (43 行)	POST <code>/api/submissions/</code> 、 <code>/api/languages/</code>	5
Step 3 评测管理	<code>app/routers/submissions.py</code> (235 行)	GET 列表/详情、PUT rejudge	5
Step 4 用户管理	<code>app/routers/users.py</code> (235 行)	注册/登录/登出、PUT 角色	5
Step 5 评测日志	<code>app/routers/logs.py</code> (43 行)	GET 日志、PUT 可见性、GET 审计	5
Step 6 前端	<code>frontend/app.py</code> (1899 行)	—	5
AI 智能命题	<code>app/ai/</code> (607 行) + <code>routers/ai.py</code> (210 行)	配置/任务/进度/取消	10

1.5 统一响应结构

所有 API 返回统一的 JSON 结构: `{code, msg, data}`。HTTP 状态码语义: `200` 正常 / `400` 参数错 / `401` 未登录 / `403` 权限不足或 banned / `404` 不存在 / `409` 冲突 / `429` 频率超限 / `500` 服务器异常。响应处理顺序为 `401` → `403` → `400` → `429` → `409` → `404` → `500`。

2. 关键实现与难点

2.1 异步评测引擎（核心难点）

异步评测是本系统最复杂、最容易出错的部分。流程如下图：

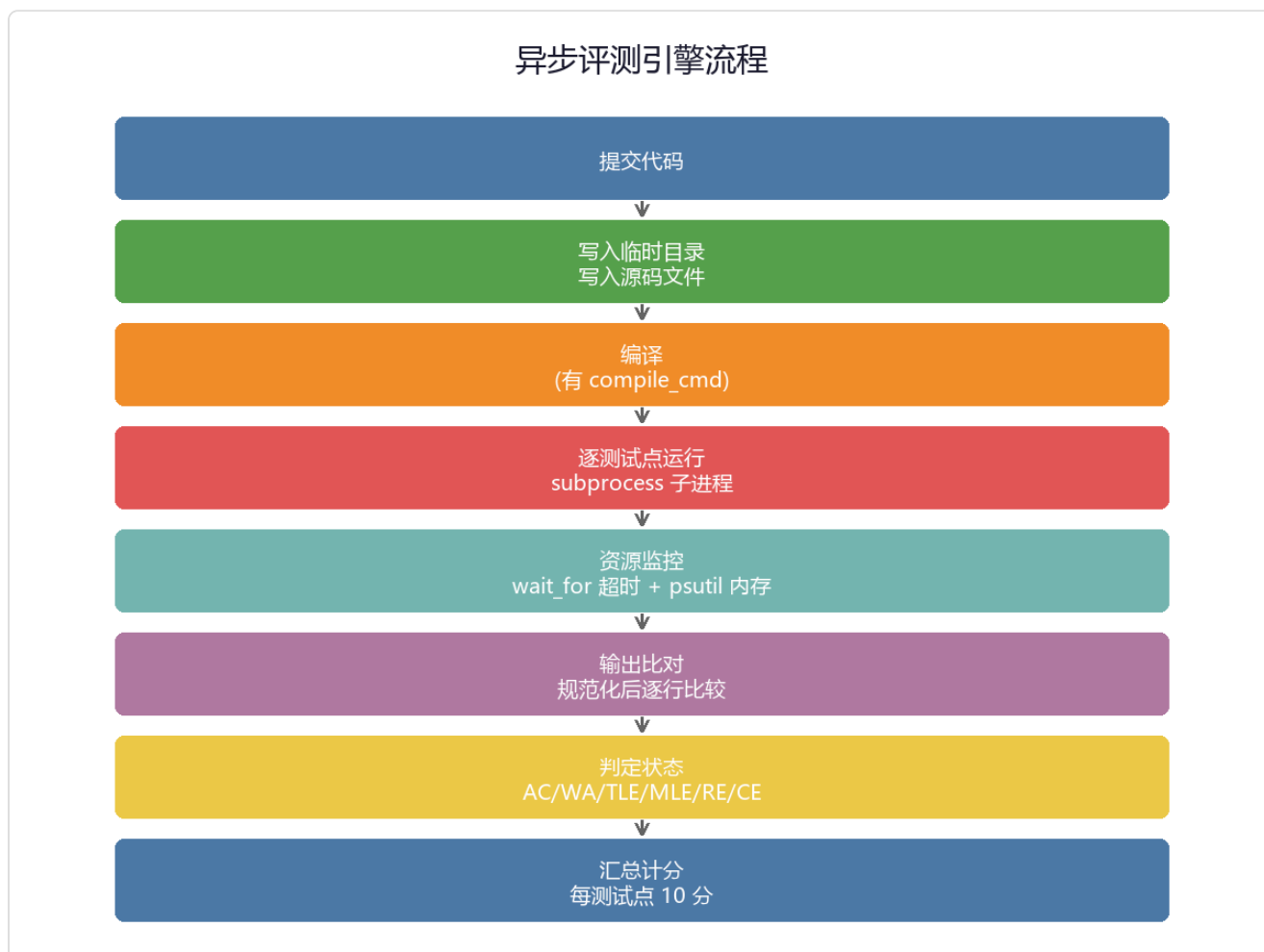


图 2.1 异步评测引擎流程

关键设计点：

- **异步执行**： `asyncio.create_subprocess_exec` 拉起子进程，主线程不被阻塞，可并发处理多份提交。
- **时间限制**： `asyncio.wait_for(proc.communicate(...), timeout=time_limit)` 强制中断，超时后调用 `_kill` 终止整个进程树（含子进程）。
- **内存限制**： `psutil` 后台协程每 50ms 轮询 RSS，超限立即杀进程并置标志位。Windows 没有 Linux 的 `resource` / `ulimit`，这是 `psutil` 方案的优势。
- **状态判定**： `_classify` 按 `timed_out` → `memory_exceeded` → `returncode` → `UNK` → `AC` → `WA` 优先级判定，编译失败时整提交标记为 `CE`。
- **输出比对**： `_normalize_output` 统一处理 `\r\n` 换行、行尾空格、尾部空行，避免 Windows 评测踩坑。

2.2 评测进程回收与内存采样（实战踩坑）

问题一：Windows 下评测队列永久卡死。超时后 `finally` 块里无界 `await proc.wait()`，在 Windows Proactor 事件循环下，`communicate()` 被 `wait_for` 取消后残留未完成的 wait future，导致后续 `proc.wait()` 永久挂起，进而卡死整个单 worker 串行评测队列（后续所有提交永远 pending）。改为有界 `asyncio.wait_for(proc.wait(), timeout=5.0)` + 兜底 `_kill` 强杀。

问题二：内存峰值恒为 0。最初在进程结束后才读 RSS，但进程退出后 `memory_info().rss` 已读不到。改为独立监控协程 `_monitor_memory` 在运行期间持续采样（含子进程），记录峰值，超限立即杀进程。

2.3 路径含空格的处理（实战踩坑）

问题：项目路径为 `D:\Lai Siyu\University\Summer Semester 2025-2026\pe\bh2`，含空格。最初用 `cmd.split()` 拆分 `"python {src}"` 时，`{src}` 替换后路径中的空格把整个命令截断，`python` 找不到文件，所有提交都返回 RE。

解决方案：用 `shlex.split()` 解析命令，并在占位符替换时给路径加双引号：

```
def _quote(path: str) → str:
    return f'"{path}"'

def resolve_placeholders(cmd: str, src_path: str, exe_path: str) → str:
    return cmd.replace("{src}", _quote(src_path)).replace("{exe}", _quote(exe_path))

def _parse_command(cmd: str) → list[str]:
    return shlex.split(cmd, posix=True)
```

修复后实测 AC / WA / TLE / RE / CE 全部正确判定。

2.4 Session 认证与权限控制

使用 `starlette.middleware.sessions.SessionMiddleware`，密钥来自 `config.SESSION_SECRET_KEY`。密码用 `bcrypt` 哈希后存储（`$2b$` 开头），登录时校验。

权限控制集中在 `app/deps.py`：

- `get_current_user(request)`：从 session 取 `user_id`，查 `users` 字典返回用户对象。

- `is_admin(user)` : 判断 `role == "admin"` 。
- `check_rate_limit(user_id, problem_id)` : 单人单题维度限流, 同一用户对同一题目 1 分钟内提交 ≥ 3 次则返回 `False` (触发 429), 不同题目互不影响; 管理员免限流。

每个路由显式调用这些依赖, 避免遗漏:

```
async def create_problem(request: Request, problem: ProblemCreate):
    user = require_login_or_401(request) # 401
    if not user: return err(401, "not logged in")
    # 普通用户也能上传题目, 删除题目另需 admin
```

2.5 异步评测队列

提交接口 `POST /api/submissions/` 不会同步阻塞:

1. 写入提交 JSON, `status=pending`, 立刻返回 `submission_id` ;
2. `queue.start_worker()` 启动的 `asyncio.create_task` 后台协程从队列中取任务;
3. 调用 `judge_submission(submission)` 真正执行评测;
4. 结果原地写回提交 JSON, `status=success/error` 。

这样既能立即响应客户端, 又能避免多次提交造成 OOM。

2.6 AI 命题模块 (进阶, 核心难点)

AI 模块采用 **OpenAI 兼容 chat completions** 接口 (`httpx` 异步调用), 实测接入 **DeepSeek** 大模型, 支持任意 provider 可配置。命题流程采用 **分阶段生成**策略, 每个阶段都写入任务 JSON 的 `progress` 字段, 前端自动轮询拉取最新进度:

AI 智能命题：分阶段生成与中断机制



图 2.2 AI 命题分阶段生成与中断机制

- 分析需求**：构造 prompt（系统提示词 + 用户需求 + 可选参考题目）。
- 生成题目主体**：第一次调用 LLM，只生成题目字段（不含 testcases），控制单次输出长度。
- 解析 JSON**：兼容 ````json` 代码块包裹或裸 JSON，并对截断做兜底修复。
- 分批生成测试点**：每批让模型生成 3 个测试点，循环凑够至少 5 个，单批失败可重试而不整体失败。
- 校验补全**：检查必填字段、补充默认值（含 tags/difficulty）、生成 `public_cases` 标志。
- 表单导入题库**：前端复用题目表单展示结果，可直接或修改后导入（保留知识点标签与难度）。

难点一：大模型输出超长被截断。早期把「整题 + 测试点」一次性让模型生成，输出超过 `max_tokens` 被截断，导致 JSON 解析失败（实测两次失败：输出 4096、8191 token 均被截断）。解决思路是**分阶段生成**（主体与测试点分开调用），从根源上避免单次输出过长；同时增强 `parse_problem_json` 的截断兜底修复（顶层逗号截断 / 补闭合括号 / 逐字段回退三种策略）。改造后实测成功出题。

难点二：命题硬超时误杀。`call_llm` 外层用 `asyncio.wait_for(timeout)` 做硬截止兜底，但固定 120s 会无差别误杀「正常但慢速」的生成——实测一次「动态规划·较难」命题，阶段一成功生成题目（4477 token），阶段二生成测试点时被 120s 硬超时误判为 `TimeoutError`，且 `str(TimeoutError())` 为空串导致前端错误提示空白。解决方案三管齐下：① 硬超时放宽到 600s（`config.AI_LLM_TIMEOUT`）；② 测试点改「分批生成」（每批 3 个，单次输出短、

耗时短，从根本降低超时概率)；③ 异常处理用 `type(e).__name__` 兜底，超时给出可读提示「模型响应超时，请稍后重试」。修复后复杂题也能稳定出题。

难点三：中断要「真正终止大模型输出」。若仅把任务状态置为 `cancelled`、依赖执行循环在阶段间检查，那么进行中的模型 HTTP 调用（几十秒）不会被打断，直到该次调用返回后才退出——这不满足「中断应实际终止任务」的要求。最终方案是维护一个全局协程注册表 `_RUNNING_TASKS`（`task_id` → `asyncio.Task`），中断时调用 `task.cancel()` 触发 `CancelledError`，立即打断正在 `await` 的 `httpx` 请求并关闭底层连接。实测运行中点击中断 → 立即 `cancelled`、`result=None`，未继续跑完剩余阶段。

Token 与费用：从 API 响应 `usage` 字段读取 `prompt_tokens` / `completion_tokens`，输入输出分离计价，费用公式：

```
cost = (input_tokens / price_unit) * input_price
      + (output_tokens / price_unit) * output_price
```

计价币种可配置（USD / CNY），DeepSeek 按人民币计价，实测生成「货架寻价」二分题：输入 1150 / 输出 16142 token，费用 **¥0.223**。

配置安全：API 密钥保存在服务器级配置 `data/ai_model_config.json`，查询接口只返回 `api_key_configured: true` 布尔标志、绝不回显明文；密钥输入框留空即「不修改」，避免改单价时误清空密钥。

2.7 访问审计（评测日志可见性）

每条 `GET /api/submissions/{id}/log` 请求都记录到 `data/audit_logs.json`，包含访问者、目标提交、时间、访问状态。访问权限三级：

- 提交者本人
- 管理员
- 任意登录用户（当题目设置 `public_cases=true` 时）

2.8 存储层读缓存（性能优化）

问题：评测列表接口逐个读文件——193 条提交 × (`get_submission` + `get_problem`) 约 386 次文件 I/O + JSON 解析，实测接口耗时 **1.6 秒**，前端点开评测记录/详情要等一两秒。

解决方案：给 `_read_json` 加一层基于文件 `mtime + size` 的读缓存，文件未变化直接返回缓存；写操作通过 `os.replace` 原子替换（必然改变 `mtime`）并主动失效缓存。该方案零风险且对所有读操

作透明生效：

```
def _read_json(path: str, default: Any) → Any:
    st = os.stat(path)
    key = (st.st_mtime_ns, st.st_size)
    cached = _read_cache.get(path)
    if cached and cached[0] == key[0] and cached[1] == key[1]:
        return cached[2]          # 命中缓存，不读盘
    ...
```

效果：评测列表接口从 1.6s 降到 **0.04s** (约 40 倍)，详情接口 0.005s；题目/用户/AI 任务列表一并提速。实测提交新评测后列表 total 正确 +1、状态正确流转，缓存一致性验证通过。

2.9 pending 详情页叠列表（前端渲染残留）

问题：任务仍在评测中（pending）时，从评测列表点进详情页，详情页下方会叠加出评测列表；但直接跳转或评测完成后点进则不触发。经浏览器复现并加调试确认，**并非代码逻辑错误**（`main()` 只调用了详情渲染函数），而是 Streamlit 前端的一个渲染残留问题。

根因：pending 分支用「`time.sleep` + 无限 `st.rerun()`」轮询。Streamlit 存在已知 bug (#8360/#8599)：当 `st.rerun()` 产生比上一帧更少的元素时（从元素繁多的「列表页」跳到元素较少的「pending 详情页」），前端 React 会渲染两个相同 key 的元素，无法删除旧的列表 DOM，导致列表残留在详情页下方。这正是「只有 pending（无限轮询）才触发、success（单次渲染）不触发」的原因。

解决方案：把轮询从「无限 `st.rerun()`」改为 `st.fragment(run_every="2s")` ——fragment 只在前端定时重跑这一小块，不触发全量 rerun；等评测/任务真正结束时，fragment 内再 `st.rerun()` 一次渲染完整结果（此时元素变多，安全）：

```
if status == "pending":
    @st.fragment(run_every="2s")
    def _poll_pending():
        code2, d2, _ = api_call("GET", f"/api/submissions/{sid}")
        if code2 == 200 and d2 and d2.get("status") != "pending":
            st.rerun()          # 评测结束，元素变多，安全地全量刷新
        else:
            st.info("⌚ 评测中，正在评测，请稍候 ... ")
    _poll_pending()
```

评测详情与 AI 命题详情两处轮询均按此重构。实测 pending 详情只显示「评测中」提示不再叠列表，评测完成后自动切换到 success 详情（测试点明细正常）。

2.10 统计口径与数据一致性（按验收反馈修正）

验收前按老师同步的口径核查，发现三处统计/清理语义与预期不符，逐一修正：

修正一：通过数「一个题目贡献一次」。原实现每次满分提交都 `resolve_count += 1`，同一用户对同一题重复 AC 会重复计数。改为用户数据新增 `resolved_problems` 集合：题目首次满分时才 +1，重复 AC 不重复计数。同时做了**旧数据惰性重建**——老用户首次评测时自动扫描其历史提交，把旧的重复计数修正为「一题一次」的准确值（实测重建结果与独立扫描完全一致）。rejudge 分数回退不回退计数，保持「曾经 AC 即通过」语义。

修正二：删除题目级联清理。原实现只删题目 JSON，该题的提交记录与审计日志残留。改为级联删除：题目 JSON + 该题全部提交文件 + `audit_logs.json` 中该题记录一并清除，并从用户 `resolved_problems` 集合移除该题（`resolve_count` 保留历史贡献不递减）。边界处理：评测 worker 在评测前后双重检查题目存在性，**评测进行中删除题目时丢弃残留任务**，避免评测结果在级联删除后重新写回文件（实测无残留）。

修正三：提交频率限制改为「单人单题」。原实现按用户全局计数（1 分钟所有题合计 3 次），改为按 `(user_id, problem_id)` 维度分别计数：同一用户对同一题 1 分钟最多 3 次，换题不影响。实测同题第 4 次 429、换题 200、管理员仍免限。

3. 成果展示

3.1 接口验收（端到端测试结果）

系统启动后通过 curl 与 Python 脚本全量测试，所有功能与边界情况均符合预期：

场景	操作	结果	状态
初始管理员	admin / admintestpassword 登录	200, session 写入	通过
添加题目	POST /api/problems/	题目 JSON 落盘	通过
题目 id 冲突	重复 id 添加	409 id 已存在	通过

AC 提交	两数之和正确代码	测试点 AC，按比例得分	通过
WA 提交	输出错误结果	测试点 WA，score=0	通过
TLE 提交	死循环	所有测试点 TLE	通过
MLE 提交	分配 422MB 内存	所有测试点 MLE，进程被终止	通过
RE 提交	<code>print(1/0)</code>	所有测试点 RE	通过
CE 提交	C++ 缺分号	返回编译错误信息，状态 CE	通过
参数校验	缺必填字段	400（统一参数错误码）	通过
空样例/测试点	samples/testcases 空列表	400 拒绝（无法评测的无效题目）	通过
权限：未登录	GET /api/problems/	401 not logged in	通过
权限：非管理员	普通用户删除题目	403 permission denied	通过
权限：banned	被封禁用户登录	403 banned	通过
频率限制	同题 1 分钟内提交 ≥ 3 次	429（单人单题，换题不受影响）	通过
重新评测	admin PUT rejudge	状态回 pending，旧日志覆盖	通过
可见性配置	PUT public_cases=true	普通用户也能查日志	通过
审计日志	GET /api/logs/access/	仅管理员，记录 view_logs 访问	通过

性能优化	评测列表接口	1.6s → 0.04s (mtime 缓存)	通过
pending 详情	评测中从列表点进详情	不再叠列表 (st.fragment 轮询)	通过
通过数统计	同题重复 AC	resolve_count 只计一次 (含历史重建)	通过
删除题目	级联清理	该题提交与审计日志一并删除, 评测中任务丢弃	通过
AI 命题	真实 DeepSeek 出题	完整题目 JSON + 5 测试点, 费用 ¥0.223	通过
AI 中断	运行中 cancel	立即 cancelled, interrupted=true	通过
AI 中断边界	已完成任务 cancel	409 拒绝 (边界正确)	通过

3.2 AI 智能命题全链路 (真实 DeepSeek)

使用真实 DeepSeek 大模型 (非 mock) 验证完整命题流程, 两次成功出题:

命题需求	生成题目	难度/标签	测试点	费用
二分查找 (超市货架价格标签情境)	「货架寻价」 binary_search_price_tag	中等偏基础 / 二分查找·数组 ·lower_bound	5 个 (覆盖最小规模/目标在首尾中间/不存在/大数值)	¥0.223
动态规划·最长公共子序列	「最大连续收益」 max_contiguous_profit	中等偏基础 / 动态规划·最大子段和·线性DP	9 个	¥0.278

全链路验证要点:

- 配置 provider_url / model / api_key (密钥仅返回 api_key_configured: true 标志, 不明文泄露);
- 提交命题需求, 任务状态机 pending → running → completed, 进度实时轮询更新;

- 3. 生成的题目严格贴合输入知识点（二分查找 / 动态规划）与难度要求，样例含边界情况；
- 4. 生成结果以「添加/编辑题目」同款表单展示，可直接或修改后导入题库；
- 5. 导入题库后提交正确代码 → 评测 AC，证明 **AI 命题与题库/评测链路完整衔接**。

4. AI 使用说明

4.1 工具链

- **主力工具**：WorkBuddy 智能体（基于大模型的代码助手），承担分模块编码、调试排错、代码 review 等实现工作。
- **辅助工具**：Web 搜索（查证 Pydantic、shlex、psutil、httpx、Streamlit 用法）。
- **环境**：纯 Windows 11 + Python 3.13.12，命令行 Git Bash；后端 FastAPI + 前端 Streamlit 分离启动。

4.2 工作流

1. **需求通读**：本人先完整阅读全部 11 份需求文档，梳理评分点与功能边界。
2. **需求转化为自然语言指令**：本人将每个功能点描述为明确的目标、约束与验收标准，交给 AI 执行。
3. **AI 分模块生成**：AI 按 Step 顺序逐个实现，每个模块先写路由再写共享层，边写边由本人用 curl 实测。
4. **本人端到端验收**：每写完一块立即实测接口，发现 bug 就向 AI 复现现象并让其定位修复；AI 命题用真实 DeepSeek 跑通全链路。
5. **持续提交**：每完成一个功能点就按 Conventional Commits 规范拆分提交并 push，而非一次性提交。

4.3 Vibe Coding 代码比例

本项目的代码 **绝大部分由 AI 直接生成**（约 **90%** 以上），本人通过自然语言描述需求驱动 AI 产出，再逐段验收、反馈问题、迭代修正。实际分工如下：

环节	承担方	说明
需求拆解与描述	本人	通读 11 份要求文档，将评分点转化为可执行的自然语言需求，逐条向 AI 描述目标、边界与验收标准。
代码编写	AI 为主	路由、Pydantic 模型、JSON 存储、异步评测引擎、AI 命题任务、Streamlit 前端等几乎全部由 AI 生成。

验收与测试	本人	逐项对照要求文档核对功能，用 curl / 真实 DeepSeek 出题 / 浏览器实测，发现并定位问题。
缺陷反馈与迭代	本人 + AI	本人描述 bug 现象（如“刷新回登录页”“改单价丢密钥”“输出被截断”“中断未真正停止”），AI 定位根因并修复。

也就是说，这是一次典型的 **Vibe Coding** 实践：本人扮演“产品经理 + 测试”角色，负责定义要做什么、判断做得对不对；AI 扮演“工程师”，负责具体实现。本人不逐行手写代码，但**全程主导方向、逐项验收、对最终交付质量负责**。

4.4 收获

- Vibe Coding 的核心能力不在“写代码”，而在 **把需求讲清楚、把问题定位准**：本次多个 bug（刷新会话竞态、密钥被空值覆盖、JSON 输出截断、中断未真正终止）都是靠本人准确复现现象、AI 才得以快速修复。
- AI 最容易“看似对但实际跑不起来”的代码是 **跨平台兼容、边界条件** 与 **异步时序竞态**；这些坑往往要在真实运行（而非静态阅读代码）中才会暴露。
- 把 **端到端测试** 嵌入开发流程，能在早期暴露问题，比单纯相信 AI 的“已完成”表述有效得多——每次都必须实体验收，不能只看代码。
- 诚实评估 AI 参与度很重要：AI 承担了绝大部分实现劳动，但**需求的理解、方向的判断、质量的把关**仍需本人完成，两者缺一不可。

5. 总结与建议

5.1 时间投入

阶段	耗时	内容
需求通读与方案设计	约 1.5 小时	通读 11 份需求文档，确认范围、技术选型、模块划分
基础模块实现 (Step 1-6)	约 4 小时	脚手架 + 6 个模块的代码与端到端测试
AI 智能命题模块	约 3 小时	模型调用、任务状态机、分阶段生成、Token 计费、真实出题与中断验证
前端交互与体验打磨	约 2 小时	三组页面美化、会话持久化、分页、AI 配置弹窗与历史列表
持续提交与推送	约 1 小时	82 次 Conventional Commits + push

报告与配图	约 1 小时	本文档 + 3 张配图
总计	约 12.5 小时	—

5.2 反思与收获

- **异步 + 子进程** 是本系统最有挑战的部分，掌握了 `asyncio.create_subprocess_exec`、`wait_for`、`psutil` 后台协程的协同使用。
- **异步任务的生命周期管理** 从评测队列延伸到 AI 命题：真正"中断"一个正在 `await` 网络请求的协程，需要维护协程句柄并 `cancel()`，仅改状态字段是不够的。
- **统一响应结构 + 异常处理顺序** (401→403→400→429→409→404→500) 让所有接口行为可预期，前端不需要为每个接口写特殊错误处理。
- **JSON 文件存储** 在小规模场景下简单可靠，但并发写需要原子 `rename`；读性能上，数据量过百后逐个读文件会成为瓶颈，**基于 `mtime` 的读缓存**是零风险的通用提速手段（实测 40 倍）。如果未来扩展到多实例部署，应改用 SQLite / Redis。
- **AI 命题** 是最有"未来感"的部分，题目生成后能直接进题库被评测，整个链路验证了"AI 与基础功能不割裂"。

5.3 改进建议

- 评测引擎可加入 **编译缓存**（相同源码不重复编译）；多测试点并发（`asyncio.gather`）可缩短总耗时。
- AI 命题可加入 **题面润色、测试点自动验证**（用 AI 自己写标程并跑一遍），提高生成质量与正确性。
- 用户管理可加入 **邮箱验证、密码强度策略、登录失败锁定** 等更严密的安全机制。
- 存储层可抽象为接口，**未来可平滑替换为数据库** (SQLite/PostgreSQL) 而不影响上层逻辑。
- 前端可加入 **实时评测日志流** (SSE 或 WebSocket) 替代轮询，进一步降低响应延迟。

— 报告结束 —

完整代码与提交历史见 [GitHub 仓库](#)